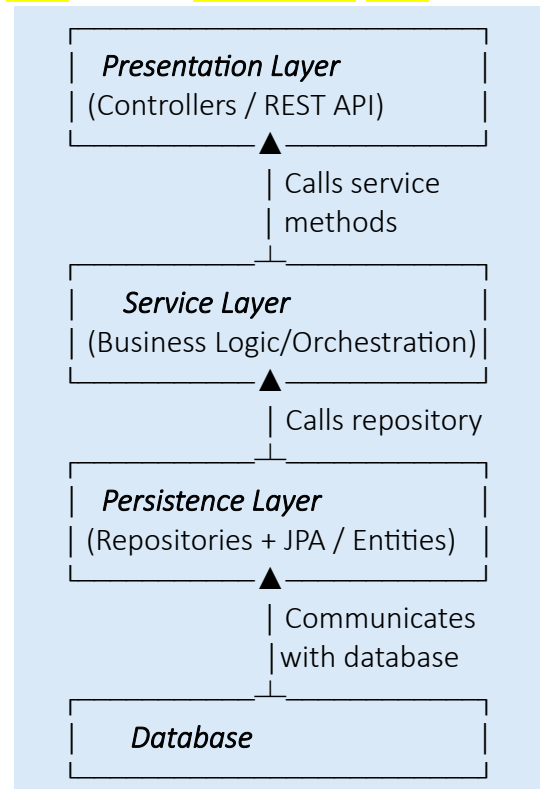


2.4 The Service Layer, Writing our Business Logic

Service Layer: The **service layer** acts as a *bridge* between the **persistence layer** and the **presentation layer**. It helps improve the *modularity*, *scalability*, and *testability* of our code.

Key roles of Service Layer:

- The *service layer* acts as a *bridge* between the *persistence layer* (responsible for data access) and the *presentation layer* (handling user interaction)
- It *encapsulates* the *business logic* of the application, *orchestrates interactions* between different *components*, and provides a *clean interface* for external clients to interact with the system.
- By *abstracting away* the complexities of *data access* and *business operations*, the service layer promotes *modularity*, *maintainability*, and *scalability*.



Service

@Service

```
public class EmployeeService {
    private final EmployeeRepository repository;
    @Autowired
    private ModelMapper mapper;
    public EmployeeService(EmployeeRepository repository) {
        this.repository = repository;
    }
    // methods
    public EmployeeDTO createNewEmployee(EmployeeDTO newEmployee) {
        EmployeeEntity employee = mapper.map(newEmployee, EmployeeEntity.class);
        EmployeeEntity createdEmployee = repository.save(employee);
        return mapper.map(createdEmployee, EmployeeDTO.class);
    }
    public List<EmployeeDTO> getAllEmployees() {
        List<EmployeeEntity> employees = repository.findAll();
        return employees
            .stream()
            .map(a -> mapper.map(a, EmployeeDTO.class))
            .collect(Collectors.toList());
    }
}
```

Controller

@RestController

@RequestMapping("/employee")

```
public class EmployeeController {
    private final EmployeeService employeeService;
    @Autowired
    private ModelMapper mapper;
    public EmployeeController(EmployeeService employeeService) {
        this.employeeService = employeeService;
    }
    @PostMapping
    public EmployeeDTO createNewEmployee(@RequestBody EmployeeDTO employee) {
        return employeeService.createNewEmployee(employee);
    }
    @GetMapping
    public List<EmployeeDTO> getAllEmployees() {
        return employeeService.getAllEmployees();
    }
}
```

In this implementation, only the *service layer interacts* directly with the *database*. The *controller* is not aware of the *entity* classes or the *repository*; it works exclusively with *DTOs*. The service layer serves as an *abstraction* between the *controller* and the *database*, ensuring a clear separation of concerns. This design improves *maintainability*, *scalability*, and *testability* by *decoupling* the *presentation* layer from the *persistence* layer.

ModelMapper :

- **ModelMapper** is a *Java library* designed to simplify the *mapping* between different *object models*, such as Entity ↔ DTO, Request ↔ Response, or any two POJOs.
- It *automates* the process of *copying data* from *one object* to *another*, reducing *boilerplate getter-setter* code and making the *mapping process* more maintainable.

Key Features :

- Automatic Mapping – **Maps fields** with *matching names* and *compatible types* automatically.
- Customizable – Supports **custom mappings** for fields with *different names* or *formats*.
- Nested Mapping – Can map **nested objects** without manual intervention.
- Type Conversion – Handles **type conversion** between fields (e.g., *String ↔ Date*) if possible.
- Partial Updates – Supports mapping into *existing objects* for **partial field updates**.

How ModelMapper Works :

ModelMapper operates by:

1. Inspecting the **Source** and **Destination** Classes

It uses **Java Reflection** to analyse the *fields* and *getter/setter* methods of both objects.

2. Matching Field **Names** and **Types**

By default, it maps fields that share:

- The **same name** (case-sensitive)
- **Compatible types** (e.g., *int* to *Integer*, *String* to *String*)

3. **Copying** Data

It transfers values from the *source object's getters* to the *destination object's setters*.

4. Handling **Complex/Nested** Mappings

If objects have *nested properties* (e.g., *address.street*), ModelMapper can map them **recursively**.

How to Use :

The *mapper.map(srcObj, dstObj)* method accepts a *source object* and a *destination object*, and maps *matching properties* from the *source* to the *destination*. This allows seamless *data transfer* between *different object types* while preserving existing destination values for fields not present in the source.

1. Basic Object to Object Mapping

```
ModelMapper mapper = new ModelMapper();  
DestinationObject dest = mapper.map(sourceObject, DestinationObject.class);
```

Creates a **new DestinationObject** and **copies** all matching fields.

2. Mapping into Existing Object

```
mapper.map(sourceObject, existingDestinationObject);
```

Updates the **existing destination** object **in place** (no *new object* is created).

Advantages :

- Eliminates repetitive *getter/setter* mapping code.
- Reduces *risk* of *human error* in manual mapping.
- Improves *readability* and *maintainability*.
- Supports advanced *mapping configurations*.